FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	CRUZ, KRISTA
Section:	AN41
Professor:	MS. TAN
Date:	10/03/2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description

- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

```
import math

case_base = [
    {
        "name": "Typhoon Yolanda (2013)",
        "problem": {"type": "typhoon", "area": "coastal", "population": 400000},
        "solution": {"strategy": "Mass coastal evacuation to reinforced centers", "evacuee_percent": 0.50,
"shelter_capacity": 500}
    },
    {
        "name": "Typhoon Rolly (2020)",
        "problem": {"type": "typhoon", "area": "rural", "population": 150000},
        "solution": {"strategy": "Evacuate agricultural zones and secure livestock", "evacuee_percent": 0.35,
"shelter_capacity": 300}
    },
    {
        "name": "Marikina River Overflow (2020)",
        "problem": {"type": "flood", "area": "urban", "population": 220000},
        "solution": {"strategy": "Riverside evacuation using local rescue boats", "evacuee_percent": 0.25,
"shelter_capacity": 350}
    },
    {
        "name": "Batangas Earthquake (2023)",
        "problem": {"type": "earthquake", "area": "rural", "population": 90000},
        "solution": {"strategy": "Open-field assembly and temporary tent setup", "evacuee_percent": 0.40,
"shelter_capacity": 200}
    }
]

new_problem = {
    "type": "typhoon",
```

```
    "area": "urban",
    "population": 180000
}

def similarity(new, old):
    score = 0.0
    if new["type"] == old["type"]:
        score += 0.5
    if new["area"] == old["area"]:
        score += 0.3
    score += 0.2 * (min(new["population"], old["population"]) / max(new["population"], old["population"]))
    return score

def adapt(new, old_case):
    old = old_case["solution"]
    evacuees = round(new["population"] * old["evacuee_percent"])
    shelters = math.ceil(evacuees / old["shelter_capacity"])
    strategy = old["strategy"]
    if new["area"] == "urban":
        strategy += "; add vertical evacuation and multi-story deployment"
    return {
        "strategy": strategy,
        "evacuee_percent": old["evacuee_percent"],
        "shelter_capacity": old["shelter_capacity"],
        "evacuees": evacuees,
        "shelters needed": shelters
    }

def main():
    print("=== CBR DISASTER RESPONSE SYSTEM ===")
    print(f"\nNEW PROBLEM: Typhoon with heavy rain in an urban area")
    print(f"Type: {new_problem['type']}, Area: {new_problem['area']}, Population:
{new_problem['population']:,}")

    print("\nSIMILARITY SCORES:")
    ranked = sorted(case_base, key=lambda c: similarity(new_problem, c["problem"]), reverse=True)
    for i, c in enumerate(ranked, 1):
        print(f" {i}. {c['name']:<32} {similarity(new_problem, c['problem']):.2f}")

    best = ranked[0]
    print(f"\nMOST SIMILAR CASE: {best['name']}")
    print(f"Old strategy: {best['solution']['strategy']}")

    final = adapt(new_problem, best)
```

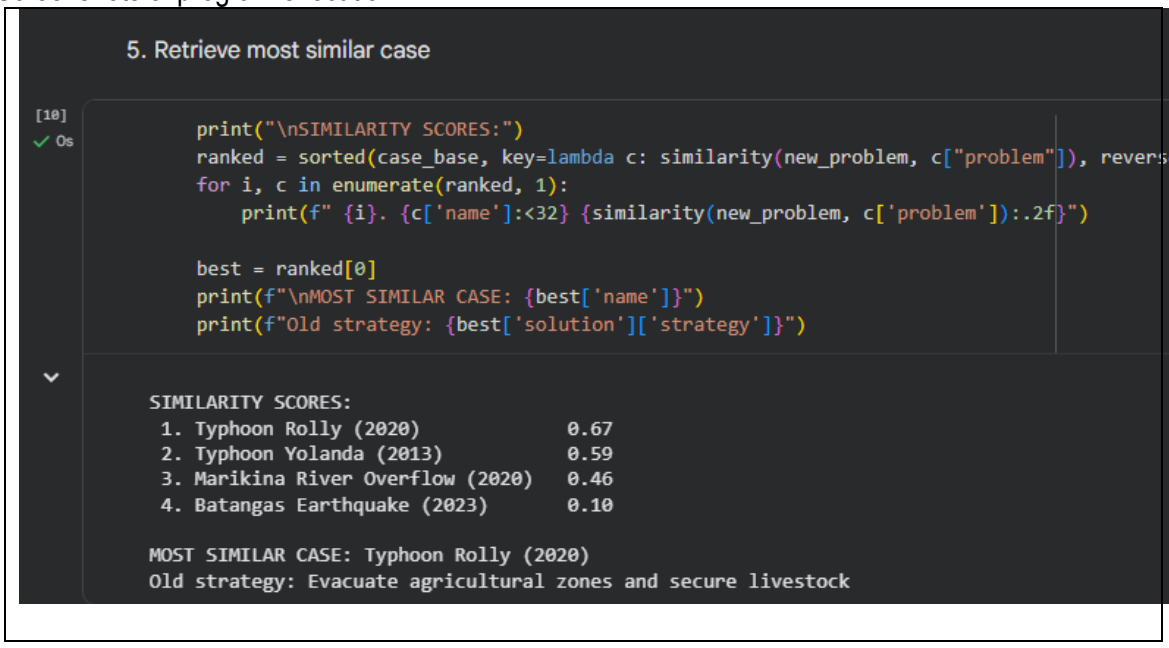
```
    print("\nADAPTED FINAL SOLUTION:")
    print(f"Strategy: {final['strategy']}")
    print(f"Evacuees: {final['evacuees']:,} ({final['evacuee_percent']:.0%} of population)")
    print(f"Shelters: {final['shelters needed']:,} (capacity {final['shelter_capacity']} each)")

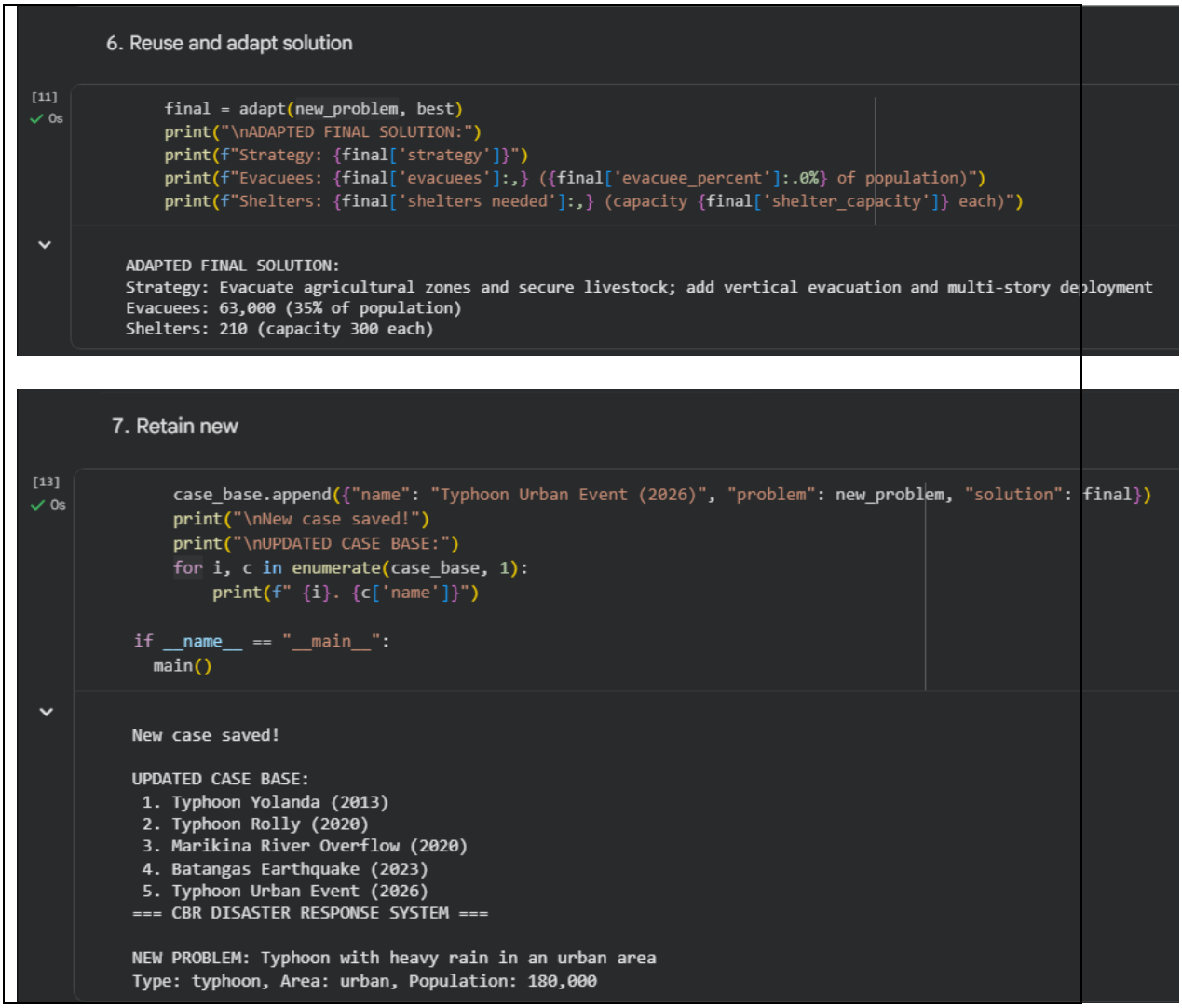
    case_base.append({"name": "Typhoon Urban Event (2026)", "problem": new_problem, "solution": final})
    print("\nNew case saved!")
    print("\nUPDATED CASE BASE:")
    for i, c in enumerate(case_base, 1):
        print(f" {i}. {c['name']}")

if __name__ == "__main__":
  main()
```

Documentation:

- Title of chosen case study
  - Disaster Response Decision Support System (DSS)
- Description of your case base
  - The case base consists of four historical disaster records: Typhoon Yolanda (2013), Typhoon Rolly (2020), Marikina River Overflow (2020), and Batangas Earthquake (2023). Each entry is formatted as a Python dictionary containing a descriptive name, problem metrics (disaster type, area classification, and population size), and a proven solution profile detailing the evacuation strategy, evacuation percentage, and individual shelter capacities.
- Explanation of similarity method
  - The program determines relevance by evaluating weighted attribute matching to score past cases against a new problem scenario on a scale from 0 to 1. Disaster type carries the highest weight (0.5) because the nature of the emergency fundamentally dictates response logistics, followed closely by area classification (0.3) for regional tactics, while population size contributes up to 0.2 based on a proportional mathematical ratio.
- Adaptation logic
  - Once the most similar past case is retrieved, the system adapts its solution for the new event. It calculates the total expected evacuees using the past evacuation percentage, divides that by the shelter capacity to find how many shelters are needed, and adds extra urban safety measures if the target area is an urban zone. The newly resolved case is then saved into the case base for future use.
- Screenshots of program execution




6. Reuse and adapt solution
[11]
0s
final = adapt(new_problem, best)
print("\nADAPTED FINAL SOLUTION:")
print(f"Strategy: {final['strategy']}")
print(f"Evacuees: {final['evacuees']:,} ({final['evacuee_percent']:.0%} of population)")
print(f"Shelters: {final['shelters needed']:,} (capacity {final['shelter_capacity']} each)")
ADAPTED FINAL SOLUTION:
Strategy: Evacuate agricultural zones and secure livestock; add vertical evacuation and multi-story deployment
Evacuees: 63,000 (35% of population)
Shelters: 210 (capacity 300 each)
7. Retain new
[13]
0s
case_base.append({"name": "Typhoon Urban Event (2026)", "problem": new_problem, "solution": final})
print("\nNew case saved!")
print("\nUPDATED CASE BASE:")
for i, c in enumerate(case_base, 1):
print(f" {i}. {c['name']}")
if __name__ == "__main__":
main()
New case saved!
UPDATED CASE BASE:
1. Typhoon Yolanda (2013)
2. Typhoon Rolly (2020)
3. Marikina River Overflow (2020)
4. Batangas Earthquake (2023)
5. Typhoon Urban Event (2026)
=== CBR DISASTER RESPONSE SYSTEM ===
NEW PROBLEM: Typhoon with heavy rain in an urban area
Type: typhoon, Area: urban, Population: 180,000

Pair Reflection:

- What each partner contributed
- What challenges you encountered
    - I encountered challenges when figuring out how to scale up the evacuation plans and shelter sizes based on the new population numbers. It took a while to ensure that the program would give realistic and logical disaster response results.
- What you learned about CBR
    - I learned that CBR is a smart way to solve problems by looking at what worked in the past. Instead of starting from scratch every time, a system can just look at a similar past event and compare it with the new event, as well as save the new event so it can be used for future references.